

DigiBank Dynamic Security Analysis

Runtime security validation of the DigiBank API.

1. Why DAST is needed

Static analysis examines source code and configuration before execution. Dynamic analysis complements it by interacting with the application while it is running. This matters because a system can compile, pass unit tests, and still expose an API route without authentication, return excessive technical detail, accept invalid input, or handle sessions incorrectly.

DAST checks what an external client can actually observe, not only what the source code appears to intend.

The sequence is: establish runtime behavior, identify security weaknesses, apply targeted remediation, and replay the same scenarios to demonstrate the result.

2. Application under test

The external client does not call each business service directly. The API Gateway is the security boundary and the single entry point for the business API. Behind it, the customer, account, transaction, compliance, and notification services run as separate Compose services.

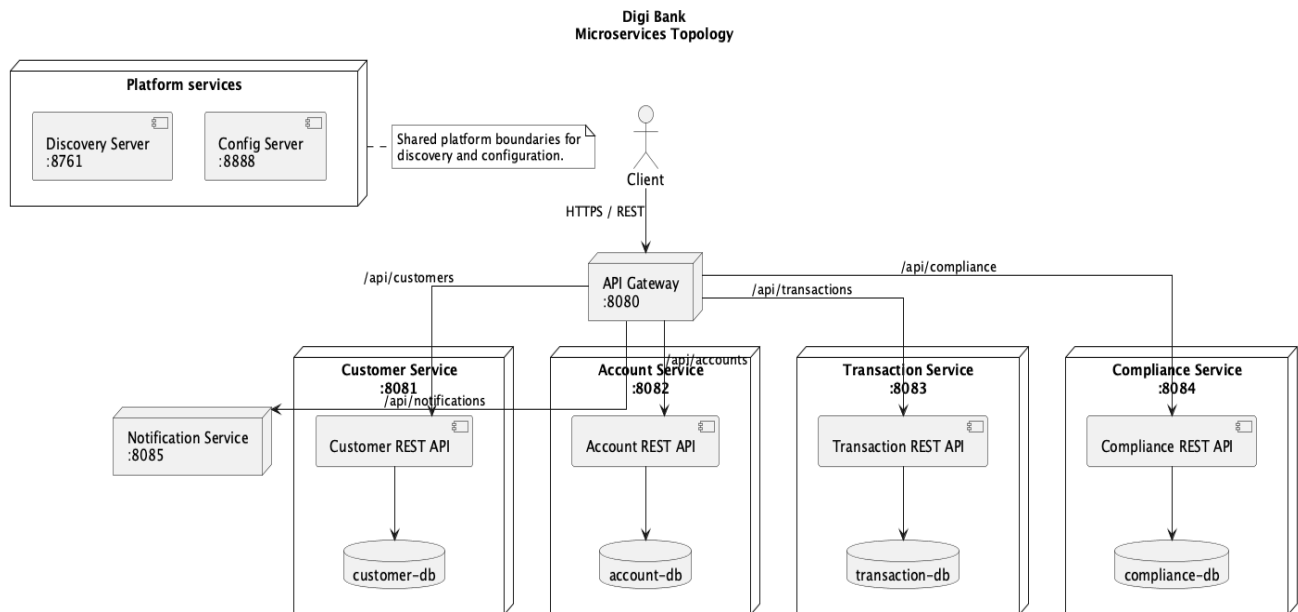


Figure 1. Runtime topology under test: gateway, business services, platform services, Compose host and persistence boundaries.

Surface	What is tested
Gateway	Authentication, authorization, headers, cache behavior, error responses and route protection.
Business APIs	Customer, account, transaction and compliance operations through gateway routing.
Runtime behavior	Malformed requests, invalid values, missing resources, session behavior and information disclosure.
Delivery path	Build, Compose startup, readiness, Newman replay and ZAP scanning.

3. Reproducible test environment

The environment is deliberately reproducible. Maven verifies the implementation, Docker Compose starts the complete topology, smoke checks verify the endpoints, and stack validation confirms that the expected containers are running before DAST begins.

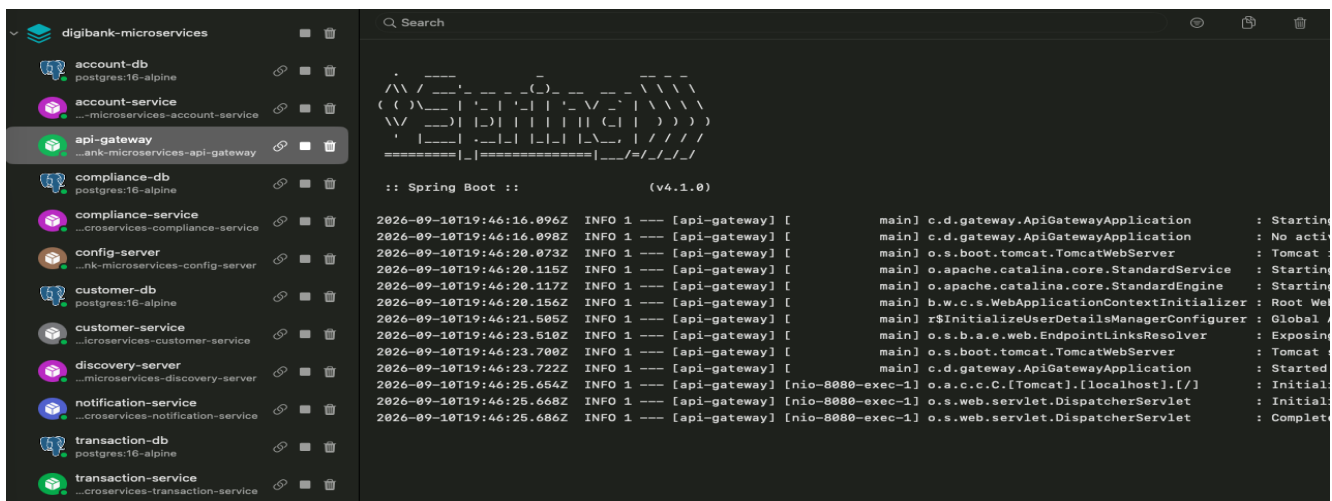
```
make clean
make build
make image-build
make up
make smoke
make validate-stack
make newman-scan
make zap-scan
```

3.1 Starting the application

The Compose startup creates the complete local runtime. The screenshot shows the command used to start the services and the resulting container state. This establishes the execution context before any security request is sent.

```
→ digibank-core git:(main) ✗ make up
/Applications/Xcode.app/Contents/Developer/usr/bin/make -C digibank-microservices up
docker compose -f docker-compose.yml up -d
[+] up 12/12
✓ Container digibank-microservices-config-server-1      Running      0.00s
✓ Container digibank-microservices-transaction-db-1    Healthy      1.25s
✓ Container digibank-microservices-compliance-db-1     Healthy      0.79s
✓ Container digibank-microservices-compliance-service-1 Running      0.00s
✓ Container digibank-microservices-account-db-1        Healthy      0.79s
✓ Container digibank-microservices-account-service-1   Running      0.00s
✓ Container digibank-microservices-notification-service-1 Running      0.00s
✓ Container digibank-microservices-transaction-service-1 Running      0.00s
✓ Container digibank-microservices-discovery-server-1  Running      0.00s
✓ Container digibank-microservices-customer-db-1       Healthy      0.79s
✓ Container digibank-microservices-customer-service-1  Running      0.00s
✓ Container digibank-microservices-api-gateway-1       Running      0.00s
```

The container view confirms that the gateway, business services, databases, discovery server and config server are present in the same runtime topology.



3.2 Smoke validation

Smoke validation checks the public health endpoint, gateway business routes, direct internal service endpoints, discovery registry and config server. It is not the security campaign; it proves that the application is alive and that later DAST failures can be interpreted as application behavior rather than startup failure.

```
→ digibank-core git:(main) ✖ make smoke
/Applications/Xcode.app/Contents/Developer/usr/bin/make -C digibank-microservices smoke
./scripts/smoke-test.sh
Checking gateway-health http://localhost:8080/actuator/health
Checking customer-api http://localhost:8080/api/customers
Checking account-api http://localhost:8080/api/accounts
Checking transaction-api http://localhost:8080/api/transactions
Checking compliance-api http://localhost:8080/api/compliance
Checking customer-service http://localhost:8081/api/customers
Checking account-service http://localhost:8082/api/accounts
Checking transaction-service http://localhost:8083/api/transactions
Checking compliance-service http://localhost:8084/api/compliance
Checking notification-service http://localhost:8085/actuator/health
Checking discovery-registry http://localhost:8761/registry
Checking config-server http://localhost:8888/config/customer-service
Microservice smoke checks passed.
```

The successful result is the gate before Newman and ZAP.

4. Baseline and remediation decision

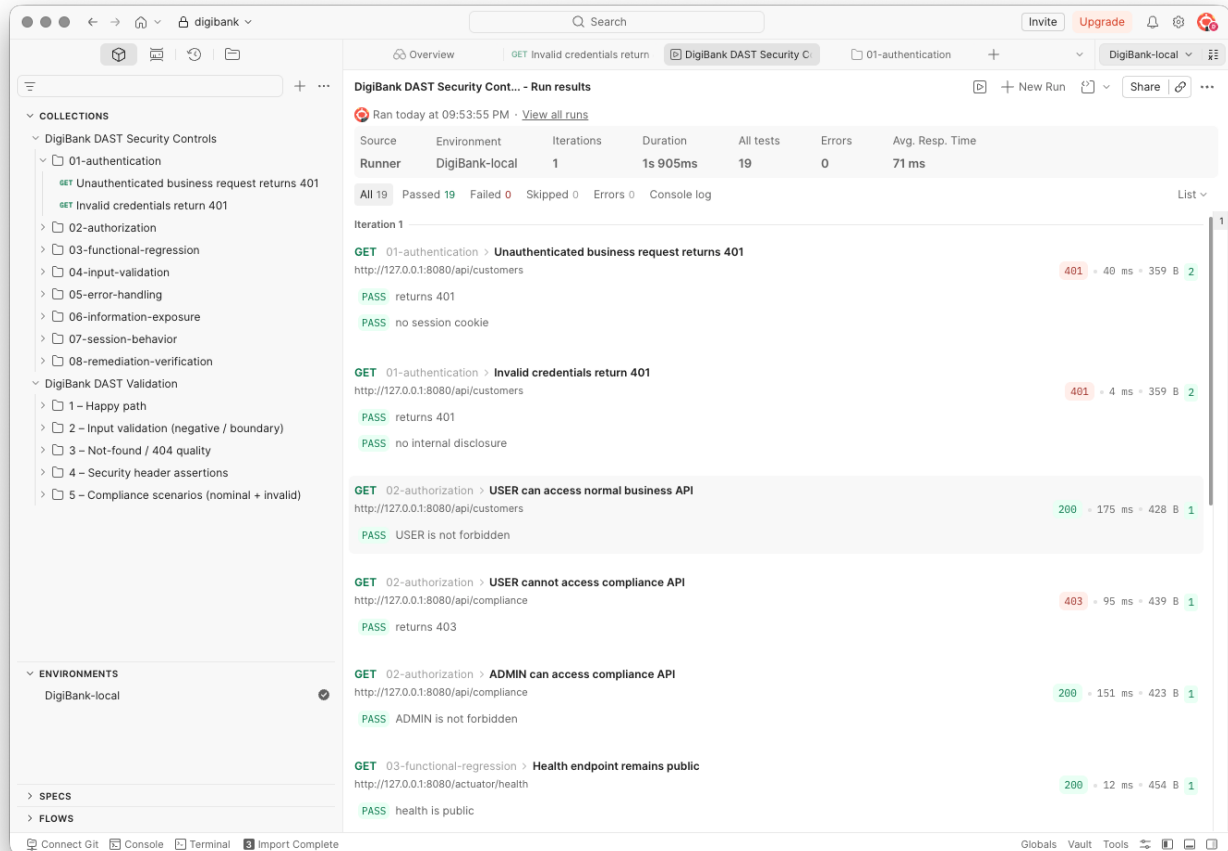
The baseline establishes the behavior before the gateway security boundary was introduced. The important observations were unauthenticated business routes, no gateway role distinction, incomplete response hardening, and no explicit cache policy for sensitive API responses.

The remediation uses stateless HTTP Basic authentication at the gateway because it is simple to reproduce in the workshop and preserves the existing service topology. It is not presented as the production identity architecture.

Request condition	Expected result after remediation
No credentials or invalid credentials	401 Unauthorized
Valid USER on normal banking API	Request is permitted
Valid USER on compliance API	403 Forbidden
Valid ADMIN on compliance API	Request is permitted
Authenticated request	No server session or session cookie

4.1 Authentication and authorization in Postman

The security collection makes the access rules visible through actual requests. The run shows unauthenticated requests returning 401, normal USER access returning 200, USER access to compliance returning 403, and ADMIN access to compliance returning 200.



The request results correspond directly to the gateway rules: authentication establishes identity, and authorization decides whether that identity may use the route.

4.2 Validation, errors and information exposure

The security collection also checks malformed JSON and missing resources. These cases return controlled 4xx responses without stack traces or internal implementation details. The same collection verifies the response headers on the health endpoint.

The screenshot displays the DigiBank DAST Security Controls interface. The left sidebar shows a tree view of collections and environments. The main panel shows the results of a test run titled 'DigiBank DAST Security Controls - Run results'. The test run was executed today at 09:53:55 PM. The summary table indicates that all 19 tests passed, with 0 errors and an average response time of 71 ms.

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	DigiBank-local	1	1s 905ms	19	0	71 ms

The test results are listed below the summary table. Each test entry includes the test name, the endpoint, the response status, the response time, and the response size. The tests are categorized by their status: PASS, FAIL, or ERROR.

- GET /127.0.0.1:8080/api/customers**: Invalid credentials return 401. Status: PASS. Response: 401, 92 ms, 447 B.
- POST /127.0.0.1:8080/api/customers**: Malformed JSON is rejected without disclosure. Status: PASS. Response: 400, 92 ms, 447 B.
- GET /127.0.0.1:8080/api/customers/999999999**: Missing resource uses controlled response. Status: PASS. Response: 404, 110 ms, 496 B.
- GET /127.0.0.1:8080/actuator/health**: Health response has security headers. Status: PASS. Response: 200, 4 ms, 454 B.
- GET /127.0.0.1:8080/api/customers**: Authenticated request is stateless. Status: PASS. Response: 200, 158 ms, 428 B.
- GET /127.0.0.1:8080/api/customers**: Authentication remains enforced after remediation. Status: PASS. Response: 401, 4 ms, 359 B.

The run shows the validation and error-handling assertions passing, including the controlled 400 and 404 responses.

4.3 Headers and stateless behavior

The headers check verifies browser and response protections. The session check verifies that an authenticated request does not create a server session or return a session cookie.

The top screenshot displays the DigiBank DAST Security Controls interface for the test 'Health response has security headers'. The test is a GET request to `{{baseUrl}}/actuator/health`. The response is a 200 OK status with a body containing JSON data:

```
1 {
2   "groups": [
3     "liveness",
4     "readiness"
5   ],
6   "status": "UP"
7 }
```

The bottom screenshot displays the DigiBank DAST Security Controls interface for the test 'Authenticated request is stateless'. The test is a GET request to `{{baseUrl}}/api/customers`. The response is a 200 OK status with a body containing JSON data:

```
1 [
2   {
3     "id": 1,
4     "firstName": "DAST",
5     "lastName": "Customer",
6     "email": "dast.1789069699173@example.com"
7   },
8   {
9     "id": 2,
10    "firstName": "DAST",
11    "lastName": "Customer",
12    "email": "dast.1789073822431@example.com"
13  }
14 ]
```

These screenshots show the two related controls: hardened responses and stateless authentication.

5. Runtime checks and scan results

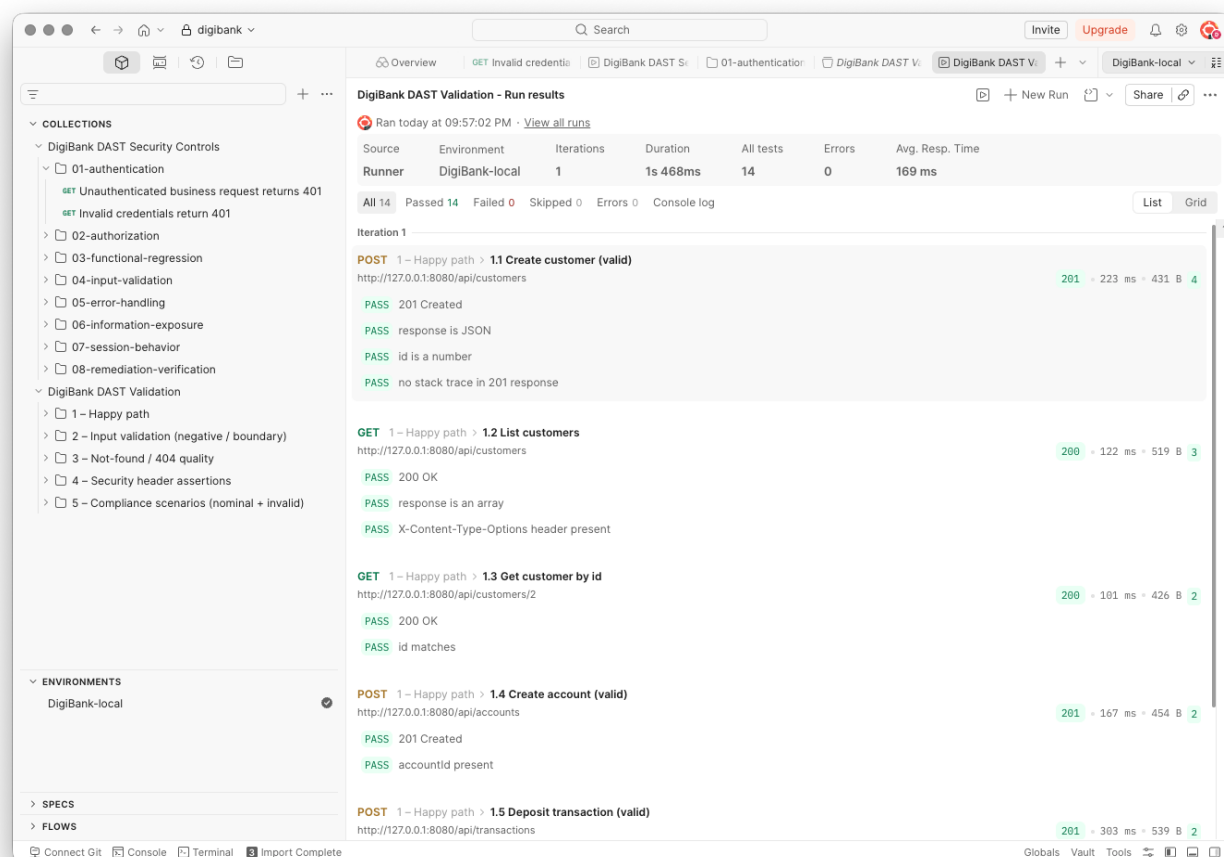
Newman expresses the expected security behavior as repeatable assertions. The security collection is divided into authentication, authorization, functional regression, input validation, error handling, information exposure, session behavior, and remediation verification.

Campaign	Result
Functional Newman collection	24 requests, 74 assertions, 0 failures.
Security Newman collection	12 requests, 19 assertions, 0 failures.
OWASP ZAP baseline	65 passing rules, 0 failures, 2 documented warnings.

The two ZAP warnings are interpreted against the local workshop environment: public non-storable resources and the plain-HTTP authentication observation. Authenticated API responses receive Cache-Control: no-store, private.

5.1 Functional regression in Postman

The functional collection checks the banking flows after the security changes: customer creation and retrieval, account creation, transactions and compliance operations. This confirms that the gateway protection did not break normal application behavior.



The collection run shows the banking requests and their passing assertions in the local environment.

5.2 Newman security replay

Newman repeats the same security collection without relying on the Postman interface. The summary is the reproducible command-line result used locally and in CI.

```
08-remediation-verification
Authentication remains enforced after remediation
GET http://127.0.0.1:8080/api/customers [401 Unauthorized, 359B, 6ms]
✓ authentication remains enforced
✓ no internal disclosure
```

	executed	failed
iterations	1	0
requests	12	0
test-scripts	12	0
prerequisite-scripts	0	0
assertions	19	0

```
total run duration: 1074ms
total data received: 1.24kB (approx)
average response time: 64ms [min: 4ms, max: 200ms, s.d.: 59ms]
[INFO] Functional report: /Users/adorsys123/dev/digibank-core/dast/reports/202609
5/functional-newman-report.json
```

The security replay completed 12 requests and 19 assertions with zero failures.

Directory listing for /

- [functional-newman-report.json](#)
- [security-newman-report.json](#)

The report directory contains separate functional and security JSON results.

5.3 OWASP ZAP scan

ZAP scans the externally visible gateway surface and complements the targeted Postman assertions. The scan checks the running application for observable response, header and authentication weaknesses that are not covered by a small set of targeted requests.

The current scan completed with zero failures and 65 passing rules. Two warnings remain: public non-storable resources and the local plain-HTTP authentication observation. They are reviewed against the local test environment rather than treated as an unexplained score.

5.4 ZAP alert summary

The ZAP summary reports zero High alerts, one Medium alert, zero Low alerts and one Informational alert. The remaining Medium alert is the local plain-HTTP warning associated with HTTP Basic authentication. It is recorded and explained in the report.

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	1
Low	0
Informational	1
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://127.0.0.1:8080	Percentage of responses with status code 2xx	50 %
Info	Informational	http://127.0.0.1:8080	Percentage of responses with status code 4xx	50 %
Info	Informational	http://127.0.0.1:8080	Percentage of endpoints with content type application/json	33 %
Info	Informational	http://127.0.0.1:8080	Percentage of endpoints with method GET	100 %
Info	Informational	http://127.0.0.1:8080	Count of total endpoints	3

5.5 Workshop 3 runtime validation

Workshop 3 validates the same security improvements against the running Docker Compose system. The sequence is intentional: rebuild the services, start the complete topology, verify readiness, replay normal and hostile HTTP scenarios, then compare the results with the earlier findings.

Runtime point	What the validation demonstrates
Authentication	Missing and invalid credentials are rejected with 401.
Authorization	USER and ADMIN roles receive different access decisions.
Validation and errors	Malformed and invalid requests return controlled 4xx responses without internal details.
Session and headers	No session cookie is created and required response protections are present.
Regression	Customer, account, transaction and compliance routes remain usable.

This is runtime evidence of the remediation, not another source-code review: the result is observed at the same gateway boundary used by an external client.

6. CI validation

The CI workflow repeats the same validation chain after the application is built. Each step has a clear purpose: start the runtime, prove readiness, verify the exposed routes, replay the security assertions, and scan the gateway surface.

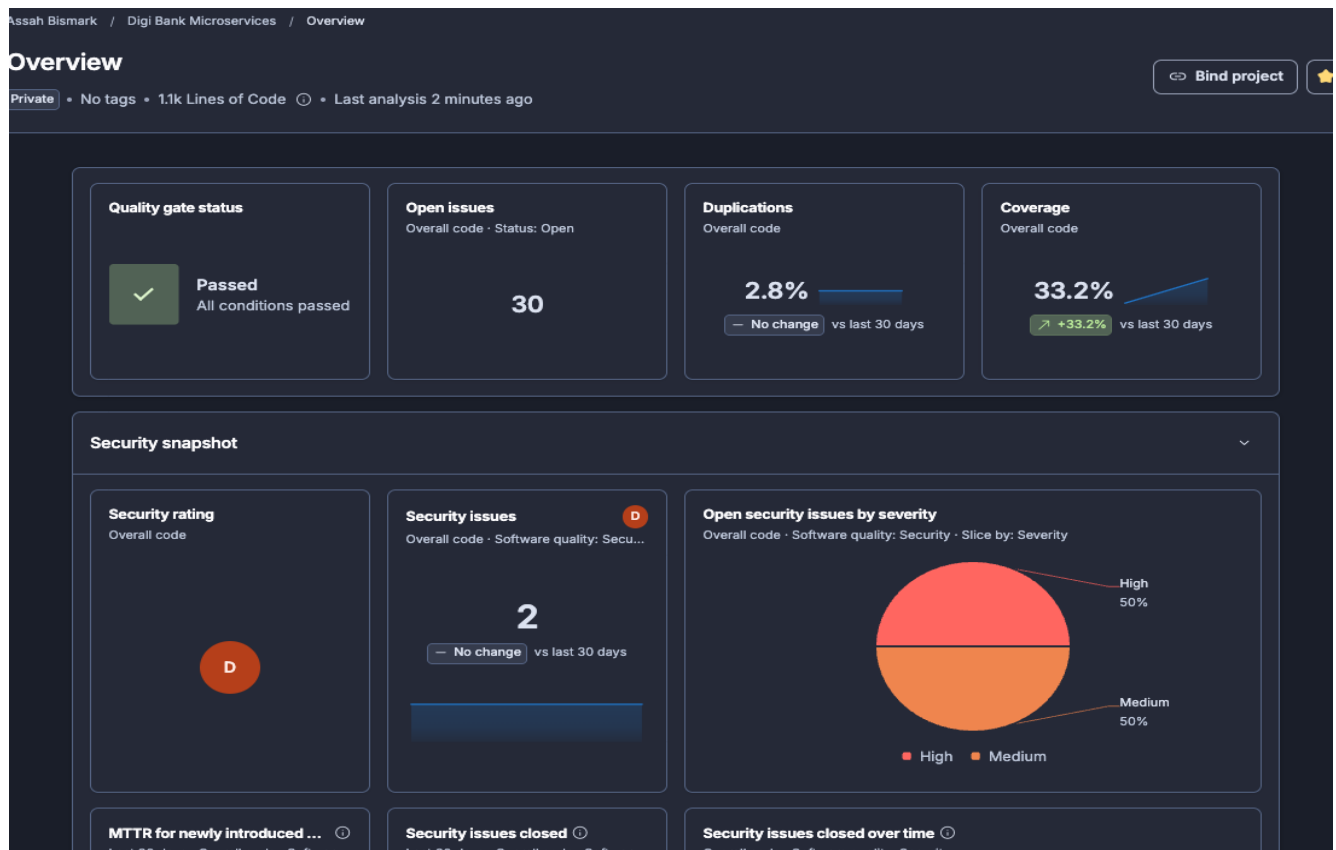
Pipeline step	Purpose
Build and tests	Compile the services and run the automated test suite.
Compose and readiness	Start the complete runtime and wait for the required endpoints.
Smoke validation	Confirm gateway, business APIs and platform services respond.
Newman	Run the versioned functional and security request assertions.
ZAP	Run an independent baseline scan against the running gateway.
Report upload	Keep Newman and ZAP results available when a check fails.

Newman and ZAP remain separate checks: Newman verifies expected security behavior, while ZAP performs broader automated observations of the exposed HTTP surface.

7. Static security analysis

Workshop 2 moves security checks earlier in the delivery lifecycle. Instead of waiting until the application is running, the source code, dependencies and test strength are inspected during the build. SonarCloud provides code and security analysis, JaCoCo supplies test coverage data, and PITest measures whether tests detect deliberately introduced defects.

The analysis is applied to the current Java 21 Spring Boot microservices implementation. It does not replace the Newman and ZAP runtime checks; it complements them.



7.1 SonarCloud findings and decisions

The open-issues view identified two security issues. The CSRF warning is an intentional decision for this workshop because the gateway uses stateless HTTP Basic authentication and does not create browser sessions or cookies. The URL-construction warning was treated as a real issue and remediated by encoding gateway path variables before forwarding them to downstream services.

sonarcloud / Digi Bank Microservices / Issues

es master

Filters 1 Sort by Priority

Applied filters (1) Status: Open x Clear filters

Select issues for bulk actions

Change 30 issues 3h 39min effort

api-gateway/.../main/java/com/digibank/gateway/SecurityConfig.java

☐ Make sure disabling Spring Security's CSRF protection is safe here. Consistency

Security High cwe former-hotspot ... +

Open Assah Bismark L20 • 5min effort • 2 hours ago

api-gateway/.../main/java/com/digibank/gateway/GatewayController.java

☐ Change this code to not construct the URL's path from user-controlled data. Intentionality

Security Medium cwe +

Open Assah Bismark L89 • 30min effort • 12 hours ago

account-service/.../com/digibank/account/error/ApiError.java

☐ Explicitly specify the time zone by passing a ZoneId or a Clock to the .now() method. Intentionality

Reliability Info Maintainability Info datetime java8 ... +

Open Assah Bismark L7 • 5min effort • 12 hours ago

api-gateway/.../main/java/com/digibank/gateway/GatewayExceptionHandler.java

Informational timestamp findings using `LocalDateTime.now()` are recorded as a maintainability follow-up. They do not represent an authorization, injection or secret-exposure defect.

7.2 JaCoCo coverage evidence

JaCoCo records which production lines are exercised by the automated tests. The report is generated during Maven verify and is imported by SonarCloud. Coverage is evidence about test reach; it is not the same as proving that the tests detect incorrect behavior.

```
➔ digibank-core git:(main) ✗ find digibank-microservices -path '*/target/site/jacoco/jacoco.xml'
```

digibank-microservices/compliance-service/target/site/jacoco/jacoco.xml
digibank-microservices/account-service/target/site/jacoco/jacoco.xml
digibank-microservices/api-gateway/target/site/jacoco/jacoco.xml
digibank-microservices/customer-service/target/site/jacoco/jacoco.xml
digibank-microservices/transaction-service/target/site/jacoco/jacoco.xml
digibank-microservices/config-server/target/site/jacoco/jacoco.xml
digibank-microservices/discovery-server/target/site/jacoco/jacoco.xml



api-gateway

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Ctx	Missed Lines	Missed Methods	Missed Classes				
com.digibank.gateway		57%		25%	20	42	30	80	13	34	0	6
Total	198 of 461	57%	12 of 16	25%	20	42	30	80	13	34	0	6



transaction-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Ctx	Missed Lines	Missed Methods	Missed Classes				
com.digibank.transaction.service		17%		0%	14	17	16	19	11	14	1	2
com.digibank.transaction.client		0%	n/a	n/a	9	9	11	11	9	9	4	4
com.digibank.transaction.error		0%	n/a	n/a	9	9	9	9	9	9	2	2
com.digibank.transaction.controller		0%	n/a	n/a	6	6	6	6	6	6	1	1
com.digibank.transaction.dto		61%	n/a	n/a	1	2	1	2	1	2	1	2
com.digibank.transaction.pattern		83%		75%	2	9	5	21	0	5	0	2
com.digibank.transaction		0%	n/a	n/a	2	2	3	3	2	2	1	1
com.digibank.transaction.model		93%	n/a	n/a	1	9	1	3	1	9	0	1
Total	419 of 582	28%	8 of 14	42%	44	63	52	74	39	56	10	15

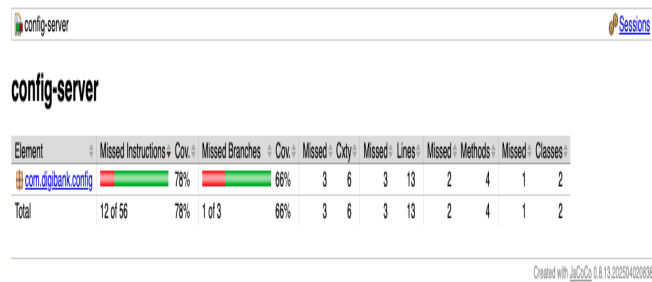
Created with JaCoCo 0.8.13.202504020838



compliance-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Ctx	Missed Lines	Missed Methods	Missed Classes				
com.digibank.compliance.service		8%		0%	10	12	7	9	9	11	0	1
com.digibank.compliance.error		0%	n/a	n/a	9	9	9	9	9	9	2	2
com.digibank.compliance.model		0%	n/a	n/a	10	10	4	4	10	10	1	1
com.digibank.compliance.dto		0%	n/a	n/a	3	3	3	3	3	3	3	3
com.digibank.compliance.controller		0%	n/a	n/a	7	7	7	7	7	7	1	1
com.digibank.compliance		0%	n/a	n/a	2	2	3	3	2	2	1	1
Total	344 of 355	3%	2 of 2	0%	41	43	33	35	40	42	8	9

Created with JaCoCo 0.8.13.202504020838



Coverage reports are available for the tested services. The notification service has no test sources and therefore produces no execution data.

7.3 PITest mutation testing

PITest changes compiled bytecode in controlled ways, such as negating a condition or replacing a return value. A killed mutant means a test detected the change. A surviving or uncovered mutant identifies a branch where the current tests are not strong enough to detect that behavior change.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
6	63% 50/80	12% 4/34	24% 4/17

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.digibank.gateway	6	63% 50/80	12% 4/34	24% 4/17

- [Project uses Spring, but the Arcmutate Spring plugin is not present.](#)

Report generated by [PIT](#) 1.17.4

Enhanced functionality available at [arcmutate.com](#)

The gateway report generated 34 mutations, killed 4, and recorded 24% test strength. This is a test-strength observation, not a failed scan.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
5	27% 12/44	10% 3/29	50% 3/6

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.digibank.customer.controller	1	0% 0/6	0% 0/5	100% 0/0
com.digibank.customer.error	2	0% 0/9	0% 0/7	100% 0/0
com.digibank.customer.model	1	64% 9/14	25% 1/4	25% 1/4
com.digibank.customer.service	1	20% 3/15	15% 2/13	100% 2/2

- [Project uses Spring, but the Arcmutate Spring plugin is not present.](#)

Report generated by [PIT](#) 1.17.4

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	100% 9/9	100% 3/3	100% 3/3

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.digibank.discovery 1	1	100% 9/9	100% 3/3	100% 3/3

-
- [Project uses Spring, but the Arcmutate Spring plugin is not present.](#)

Report generated by [PIT](#) 1.17.4

Enhanced functionality available at [arcmutate.com](#)

The other service reports show that mutation results are module-specific; they should be reviewed as test-improvement input rather than combined into one misleading score.

7.4 Static-analysis delivery controls

Static checks are exposed through simple Make commands so the instructor and the team can reproduce the same controls locally.

```
make build
make sast-sonar
make sast-pitest
make sast-dependency-check
```

The CI workflow runs SonarCloud as a separate job, runs Dependency-Check and PITest in the static-analysis job, uploads reports even when a scan fails, and caches the Dependency-Check NVD database. The first NVD database download is large; later runs reuse the GitHub Actions cache and retrieve only updates.

Control	Evidence and decision
SonarCloud	Quality gate passed; issues reviewed and classified.
JaCoCo	Coverage generated and imported into SonarCloud.
PITest	Mutation reports generated; surviving mutants become test-strength follow-up.
Dependency-Check	NVD report generated; zero unsuppressed CVEs after the Tomcat patch and reviewed false-positive rules.
CI	Separate jobs, secret-based credentials, report upload and cache reuse.

7.5 Dependency-Check results

The NVD update completed and the dependency scan generated HTML and JSON reports. The initial API Gateway report contained 43 unique CVE identifiers. After upgrading Spring Boot to 4.1.1, overriding embedded Tomcat to 11.0.25 and rerunning the scan, no unsuppressed CVEs remain across the nine Maven projects. Two dependency matches were confirmed as false positives and are covered by narrowly scoped suppression rules.

```
CVE-2026-59282, CVE-2026-47883, CVE-2026-47887, CVE-2026-59281, CVE-2026-59280, CVE-2026-593
tomcat-embed-core-11.0.22.jar (pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@11.0.22, d
2.3:a:apache:tomcat:11.0.22:*:*:*:*:*:, cpe:2.3:a:apache:tomcat:apache_tomcat:11.0.22:*:*
+*:*:**) : CVE-2026-65637, CVE-2026-65905, CVE-2026-53434, CVE-2026-55276, CVE-2026-59083, C
-2026-59084, CVE-2026-65182, CVE-2026-68525, CVE-2026-65183, CVE-2026-66422, CVE-2026-68569,
E-2026-65927, CVE-2026-68763, CVE-2026-53404, CVE-2026-73180, CVE-2026-55955, CVE-2026-55956,
VE-2026-50229, CVE-2026-66299
See the dependency-check report for more details.

[INFO] -----
[INFO] Reactor Summary for Digi Bank Microservices 1.0.0-SNAPSHOT:
[INFO] Digi Bank Microservices ..... SUCCESS [ 5.547 s]
[INFO] api-gateway ..... SUCCESS [ 5.238 s]
[INFO] discovery-server ..... SUCCESS [ 20.086 s]
[INFO] config-server ..... SUCCESS [ 5.298 s]
[INFO] customer-service ..... SUCCESS [ 5.237 s]
[INFO] account-service ..... SUCCESS [ 4.749 s]
[INFO] transaction-service ..... SUCCESS [ 4.345 s]
[INFO] compliance-service ..... SUCCESS [ 3.651 s]
[INFO] notification-service ..... SUCCESS [ 2.484 s]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 56.875 s
[INFO] Finished at: 2026-09-11T09:40:59+01:00
[INFO] -----
```



DEPENDENCY-CHECK

Dependency-Check is an open source tool performing local audit analysis of all your dependencies. Most products and libraries register their use to an available identifier on the Internet. One of the first goals of the reporting provided facilitates acceptance for use in a production environment, and there are not identifiers, linked or otherwise, with regard to the identifier or its use. As of this point and the reporting provided to the client's user, it is meant that the copyright holder or the build tool for any dependencies released under the license.

[How to read the report](#) | [Suppressing false positives](#) | [Getting Help: github issues](#)

🔒 [Sponsor](#)

Project: notification-service

com.digibank.microservices.notification-service:1.0.0-SNAPSHOT

Scan Information (show all):

- dependency-check version: 12.1.0
- Report Generated On: Fri, 11 Sep 2020 08:40:59 +0100
- Dependencies Scanned: 50 (27 unique)
- Vulnerable Dependencies: 5
- Vulnerabilities Found: 5
- Vulnerabilities Suppressed: 0
- ...

Summary

Display: [Showing Vulnerable Dependencies \(click to show all\)](#)

Dependency	Vulnerability Ids	Package	Highest Severity	CVE Count	Confidence	Evidence
hibernate-validator: 5.1.2.Final.jar	cpe:2.3:hibernate:hibernate-validator:5.1.0:***** cpe:2.3:hibernate:hibernate-validator:5.1.0:***** cpe:2.3:hibernate-validator:5.1.0:*****	pkg:maven/org.hibernate.validator/hibernate-validator@5.1.0.Final	MEDIUM	1	Highest	78
log4j-core: 2.23.4.jar	cpe:2.3:apache:log4j:2.25.4:*****	pkg:maven/org.apache.logging.log4j/log4j-core@2.25.4	MEDIUM	1	Highest	43
spring-core: 7.0.8.jar	cpe:2.3:spring:springframework:7.0.8:***** cpe:2.3:springframework:springframework:7.0.8:***** cpe:2.3:springframework:springframework:7.0.8:*****	pkg:maven/org.springframework/spring-core@7.0.8	CRITICAL	17	Highest	41
spring-web: 7.0.8.jar	cpe:2.3:spring:springframework:7.0.8:***** cpe:2.3:springframework:springframework:7.0.8:***** cpe:2.3:springframework:springframework:7.0.8:***** cpe:2.3:web_project:web:7.0.8:*****	pkg:maven/org.springframework/spring-web@7.0.8	CRITICAL	17	Highest	35
tomcat-embed-core: 11.0.22.jar	cpe:2.3:apache:tomcat:11.0.22:***** cpe:2.3:tomcat:tomcat:11.0.22:*****	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@11.0.22	CRITICAL	19	Highest	63

DEPENDENCY-CHECK

[How to read the report](#) | [Suppressing false positives](#) | **Getting Help:** [github issues](#)

 [Sponsor](#)

Project: notification-service

com.digibank.microservices:notification-service:1.0.0-SNAPSHOT

Scan information ([show all](#))

- dependency-check version: 12.1.0
- Report Generated On: Fri, 11 Sep 2026 09:40:59 +0100
- Dependencies Scanned: 55 (27 unique)
- Vulnerable Dependencies: 5
- Vulnerabilities Found: 55
- Vulnerabilities Suppressed: 0

Summary

Display: [Showing Vulnerable Dependencies \(click to show all\)](#)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence
hibernate: 5.2.5.Final	CVE-2017-7500 CVE-2017-7501 CVE-2017-7502 CVE-2017-7503 CVE-2017-7504 CVE-2017-7505 CVE-2017-7506 CVE-2017-7507 CVE-2017-7508 CVE-2017-7509 CVE-2017-7510 CVE-2017-7511 CVE-2017-7512 CVE-2017-7513 CVE-2017-7514 CVE-2017-7515 CVE-2017-7516 CVE-2017-7517 CVE-2017-7518 CVE-2017-7519 CVE-2017-7520 CVE-2017-7521 CVE-2017-7522 CVE-2017-7523 CVE-2017-7524 CVE-2017-7525 CVE-2017-7526 CVE-2017-7527 CVE-2017-7528 CVE-2017-7529 CVE-2017-7530 CVE-2017-7531 CVE-2017-7532 CVE-2017-7533 CVE-2017-7534 CVE-2017-7535 CVE-2017-7536 CVE-2017-7537 CVE-2017-7538 CVE-2017-7539 CVE-2017-7540 CVE-2017-7541 CVE-2017-7542 CVE-2017-7543 CVE-2017-7544 CVE-2017-7545 CVE-2017-7546 CVE-2017-7547 CVE-2017-7548 CVE-2017-7549 CVE-2017-7550 CVE-2017-7551 CVE-2017-7552 CVE-2017-7553 CVE-2017-7554 CVE-2017-7555 CVE-2017-7556 CVE-2017-7557 CVE-2017-7558 CVE-2017-7559 CVE-2017-7560 CVE-2017-7561 CVE-2017-7562 CVE-2017-7563 CVE-2017-7564 CVE-2017-7565 CVE-2017-7566 CVE-2017-7567 CVE-2017-7568 CVE-2017-7569 CVE-2017-7570 CVE-2017-7571 CVE-2017-7572 CVE-2017-7573 CVE-2017-7574 CVE-2017-7575 CVE-2017-7576 CVE-2017-7577 CVE-2017-7578 CVE-2017-7579 CVE-2017-7580 CVE-2017-7581 CVE-2017-7582 CVE-2017-7583 CVE-2017-7584 CVE-2017-7585 CVE-2017-7586 CVE-2017-7587 CVE-2017-7588 CVE-2017-7589 CVE-2017-7590 CVE-2017-7591 CVE-2017-7592 CVE-2017-7593 CVE-2017-7594 CVE-2017-7595 CVE-2017-7596 CVE-2017-7597 CVE-2017-7598 CVE-2017-7599 CVE-2017-7600 CVE-2017-7601 CVE-2017-7602 CVE-2017-7603 CVE-2017-7604 CVE-2017-7605 CVE-2017-7606 CVE-2017-7607 CVE-2017-7608 CVE-2017-7609 CVE-2017-7610 CVE-2017-7611 CVE-2017-7612 CVE-2017-7613 CVE-2017-7614 CVE-2017-7615 CVE-2017-7616 CVE-2017-7617 CVE-2017-7618 CVE-2017-7619 CVE-2017-7620 CVE-2017-7621 CVE-2017-7622 CVE-2017-7623 CVE-2017-7624 CVE-2017-7625 CVE-2017-7626 CVE-2017-7627 CVE-2017-7628 CVE-2017-7629 CVE-2017-7630 CVE-2017-7631 CVE-2017-7632 CVE-2017-7633 CVE-2017-7634 CVE-2017-7635 CVE-2017-7636 CVE-2017-7637 CVE-2017-7638 CVE-2017-7639 CVE-2017-7640 CVE-2017-7641 CVE-2017-7642 CVE-2017-7643 CVE-2017-7644 CVE-2017-7645 CVE-2017-7646 CVE-2017-7647 CVE-2017-7648 CVE-2017-7649 CVE-2017-7650 CVE-2017-7651 CVE-2017-7652 CVE-2017-7653 CVE-2017-7654 CVE-2017-7655 CVE-2017-7656 CVE-2017-7657 CVE-2017-7658 CVE-2017-7659 CVE-2017-7660 CVE-2017-7661 CVE-2017-7662 CVE-2017-7663 CVE-2017-7664 CVE-2017-7665 CVE-2017-7666 CVE-2017-7667 CVE-2017-7668 CVE-2017-7669 CVE-2017-7670 CVE-2017-7671 CVE-2017-7672 CVE-2017-7673 CVE-2017-7674 CVE-2017-7675 CVE-2017-7676 CVE-2017-7677 CVE-2017-7678 CVE-2017-7679 CVE-2017-7680 CVE-2017-7681 CVE-2017-7682 CVE-2017-7683 CVE-2017-7684 CVE-2017-7685 CVE-2017-7686 CVE-2017-7687 CVE-2017-7688 CVE-2017-7689 CVE-2017-7690 CVE-2017-7691 CVE-2017-7692 CVE-2017-7693 CVE-2017-7694 CVE-2017-7695 CVE-2017-7696 CVE-2017-7697 CVE-2017-7698 CVE-2017-7699 CVE-2017-7700 CVE-2017-7701 CVE-2017-7702 CVE-2017-7703 CVE-2017-7704 CVE-2017-7705 CVE-2017-7706 CVE-2017-7707 CVE-2017-7708 CVE-2017-7709 CVE-2017-7710 CVE-2017-7711 CVE-2017-7712 CVE-2017-7713 CVE-2017-7714 CVE-2017-7715 CVE-2017-7716 CVE-2017-7717 CVE-2017-7718 CVE-2017-7719 CVE-2017-7720 CVE-2017-7721 CVE-2017-7722 CVE-2017-7723 CVE-2017-7724 CVE-2017-7725 CVE-2017-7726 CVE-2017-7727 CVE-2017-7728 CVE-2017-7729 CVE-2017-7730 CVE-2017-7731 CVE-2017-7732 CVE-2017-7733 CVE-2017-7734 CVE-2017-7735 CVE-2017-7736 CVE-2017-7737 CVE-2017-7738 CVE-2017-7739 CVE-2017-7740 CVE-2017-7741 CVE-2017-7742 CVE-2017-7743 CVE-2017-7744 CVE-2017-7745 CVE-2017-7746 CVE-2017-7747 CVE-2017-7748 CVE-2017-7749 CVE-2017-7750 CVE-2017-7751 CVE-2017-7752 CVE-2017-7753 CVE-2017-7754 CVE-2017-7755 CVE-2017-7756 CVE-2017-7757 CVE-2017-7758 CVE-2017-7759 CVE-2017-7760 CVE-2017-7761 CVE-2017-7762 CVE-2017-7763 CVE-2017-7764 CVE-2017-7765 CVE-2017-7766 CVE-2017-7767 CVE-2017-7768 CVE-2017-7769 CVE-2017-7770 CVE-2017-7771 CVE-2017-7772 CVE-2017-7773 CVE-2017-7774 CVE-2017-7775 CVE-2017-7776 CVE-2017-7777 CVE-2017-7778 CVE-2017-7779 CVE-2017-7780 CVE-2017-7781 CVE-2017-7782 CVE-2017-7783 CVE-2017-7784 CVE-2017-7785 CVE-2017-7786 CVE-2017-7787 CVE-2017-7788 CVE-2017-7789 CVE-2017-7790 CVE-2017-7791 CVE-2017-7792 CVE-2017-7793 CVE-2017-7794 CVE-2017-7795 CVE-2017-7796 CVE-2017-7797 CVE-2017-7798 CVE-2017-7799 CVE-2017-7800 CVE-2017-7801 CVE-2017-7802 CVE-2017-7803 CVE-2017-7804 CVE-2017-7805 CVE-2017-7806 CVE-2017-7807 CVE-2017-7808 CVE-2017-7809 CVE-2017-7810 CVE-2017-7811 CVE-2017-7812 CVE-2017-7813 CVE-2017-7814 CVE-2017-7815 CVE-2017-7816 CVE-2017-7817 CVE-2017-7818 CVE-2017-7819 CVE-2017-7820 CVE-2017-7821 CVE-2017-7822 CVE-2017-7823 CVE-2017-7824 CVE-2017-7825 CVE-2017-7826 CVE-2017-7827 CVE-2017-7828 CVE-2017-7829 CVE-2017-7830 CVE-2017-7831 CVE-2017-7832 CVE-2017-7833 CVE-2017-7834 CVE-2017-7835 CVE-2017-7836 CVE-2017-7837 CVE-2017-7838 CVE-2017-7839 CVE-2017-7840 CVE-2017-7841 CVE-2017-7842 CVE-2017-7843 CVE-2017-7844 CVE-2017-7845 CVE-2017-7846 CVE-2017-7847 CVE-2017-7848 CVE-2017-7849 CVE-2017-7850 CVE-2017-7851 CVE-2017-7852 CVE-2017-7853 CVE-2017-7854 CVE-2017-7855 CVE-2017-7856 CVE-2017-7857 CVE-2017-7858 CVE-2017-7859 CVE-2017-7860 CVE-2017-7861 CVE-2017-7862 CVE-2017-7863 CVE-2017-7864 CVE-2017-7865 CVE-2017-7866 CVE-2017-7867 CVE-2017-7868 CVE-2017-7869 CVE-2017-7870 CVE-2017-7871 CVE-2017-7872 CVE-2017-7873 CVE-2017-7874 CVE-2017-7875 CVE-2017-7876 CVE-2017-7877 CVE-2017-7878 CVE-2017-7879 CVE-2017-7880 CVE-2017-7881 CVE-2017-7882 CVE-2017-7883 CVE-2017-7884 CVE-2017-7885 CVE-2017-7886 CVE-2017-7887 CVE-2017-7888 CVE-2017-7889 CVE-2017-7890 CVE-2017-7891 CVE-2017-7892 CVE-2017-7893 CVE-2017-7894 CVE-2017-7895 CVE-2017-7896 CVE-2017-7897 CVE-2017-7898 CVE-2017-7899 CVE-2017-7900 CVE-2017-7901 CVE-2017-7902 CVE-2017-7903 CVE-2017-7904 CVE-2017-7905 CVE-2017-7906 CVE-2017-7907 CVE-2017-7908 CVE-2017-7909 CVE-2017-7910 CVE-2017-7911 CVE-2017-7912 CVE-2017-7913 CVE-2017-7914 CVE-2017-7915 CVE-2017-7916 CVE-2017-7917 CVE-2017-7918 CVE-2017-7919 CVE-2017-7920 CVE-2017-7921 CVE-2017-7922 CVE-2017-7923 CVE-2017-7924 CVE-2017-7925 CVE-2017-7926 CVE-2017-7927 CVE-2017-7928 CVE-2017-7929 CVE-2017-7930 CVE-2017-7931 CVE-2017-7932 CVE-2017-7933 CVE-2017-7934 CVE-2017-7935 CVE-2017-7936 CVE-2017-7937 CVE-2017-7938 CVE-2017-7939 CVE-2017-7940 CVE-2017-7941 CVE-2017-7942 CVE-2017-7943 CVE-2017-7944 CVE-2017-7945 CVE-2017-7946 CVE-2017-7947 CVE-2017-7948 CVE-2017-7949 CVE-2017-7950 CVE-2017-7951 CVE-2017-7952 CVE-2017-7953 CVE-2017-7954 CVE-2017-7955 CVE-2017-7956 CVE-2017-7957 CVE-2017-7958 CVE-2017-7959 CVE-2017-7960 CVE-2017-7961 CVE-2017-7962 CVE-2017-7963 CVE-2017-7964 CVE-2017-7965 CVE-2017-7966 CVE-2017-7967 CVE-2017-7968 CVE-2017-7969 CVE-2017-7970 CVE-2017-7971 CVE-2017-7972 CVE-2017-7973 CVE-2017-7974 CVE-2017-7975 CVE-2017-7976 CVE-2017-7977 CVE-2017-7978 CVE-2017-7979 CVE-2017-7980 CVE-2017-7981 CVE-2017-7982 CVE-2017-7983 CVE-2017-7984 CVE-2017-7985 CVE-2017-7986 CVE-2017-7987 CVE-2017-7988 CVE-2017-7989 CVE-2017-7990 CVE-2017-7991 CVE-2017-7992 CVE-2017-7993 CVE-2017-7994 CVE-2017-7995 CVE-2017-7996 CVE-2017-7997 CVE-2017-7998 CVE-2017-7999 CVE-2017-8000 CVE-2017-8001 CVE-2017-8002 CVE-2017-8003 CVE-2017-8004 CVE-2017-8005 CVE-2017-8006 CVE-2017-8007 CVE-2017-8008 CVE-2017-8009 CVE-2017-8010 CVE-2017-8011 CVE-2017-8012 CVE-2017-8013 CVE-2017-8014 CVE-2017-8015 CVE-2017-8016 CVE-2017-8017 CVE-2017-8018 CVE-2017-8019 CVE-2017-8020 CVE-2017-8021 CVE-2017-8022 CVE-2017-8023 CVE-2017-8024 CVE-2017-8025 CVE-2017-8026 CVE-2017-8027 CVE-2017-8028 CVE-2017-8029 CVE-2017-8030 CVE-2017-8031 CVE-2017-8032 CVE-2017-8033 CVE-2017-8034 CVE-2017-8035 CVE-2017-8036 CVE-2017-8037 CVE-2017-8038 CVE-2017-8039 CVE-2017-8040 CVE-2017-8041 CVE-2017-8042 CVE-2017-8043 CVE-2017-8044 CVE-2017-8045 CVE-2017-8046 CVE-2017-8047 CVE-2017-8048 CVE-2017-8049 CVE-2017-8050 CVE-2017-8051 CVE-2017-8052 CVE-2017-8053 CVE-2017-8054 CVE-2017-8055 CVE-2017-8056 CVE-2017-8057 CVE-2017-8058 CVE-2017-8059 CVE-2017-8060 CVE-2017-8061 CVE-2017-8062 CVE-2017-8063 CVE-2017-8064 CVE-2017-8065 CVE-2017-8066 CVE-2017-8067 CVE-2017-8068 CVE-2017-8069 CVE-2017-8070 CVE-2017-8071 CVE-2017-8072 CVE-2017-8073 CVE-2017-8074 CVE-2017-8075 CVE-2017-8076 CVE-2017-8077 CVE-2017-8078 CVE-2017-8079 CVE-2017-8080 CVE-2017-8081 CVE-2017-8082 CVE-2017-8083 CVE-2017-8084 CVE-2017-8085 CVE-2017-8086 CVE-2017-8087 CVE-2017-8088 CVE-2017-8089 CVE-2017-8090 CVE-2017-8091 CVE-2017-8092 CVE-2017-8093 CVE-2017-8094 CVE-2017-8095 CVE-2017-8096 CVE-2017-8097 CVE-2017-8098 CVE-2017-8099 CVE-2017-8100 CVE-2017-8101 CVE-2017-8102 CVE-2017-8103 CVE-2017-8104 CVE-2017-8105 CVE-2017-8106 CVE-2017-8107 CVE-2017-8108 CVE-2017-8109 CVE-2017-8110 CVE-2017-8111 CVE-2017-8112 CVE-2017-8113 CVE-2017-8114 CVE-2017-8115 CVE-2017-8116 CVE-2017-8117 CVE-2017-8118 CVE-2017-8119 CVE-2017-8120 CVE-2017-8121 CVE-2017-8122 CVE-2017-8123 CVE-2017-8124 CVE-2017-8125 CVE-2017-8126 CVE-2017-8127 CVE-2017-8128 CVE-2017-8129 CVE-2017-8130 CVE-2017-8131 CVE-2017-8132 CVE-2017-8133 CVE-2017-8134 CVE-2017-8135 CVE-2017-8136 CVE-2017-8137 CVE-2017-8138 CVE-2017-8139 CVE-2017-8140 CVE-2017-8141 CVE-2017-8142 CVE-2017-8143 CVE-2017-8144 CVE-2017-8145 CVE-2017-8146 CVE-2017-8147 <					

The captured NVD run shows the completed update, while the Dependency-Check report shows the scan output. The NIST source reference is retained as supporting context for the vulnerability database used by the scan.

The image is a screenshot of the NIST National Vulnerability Database (NVD) website. At the top, there is a dark blue header with the NIST logo on the left and a 'NVD MENU' button on the right. Below the header, there is a blue banner with the text 'Information Technology Laboratory' and 'NATIONAL VULNERABILITY DATABASE'. The main content area has a white background and features the heading 'Generating Your API Key' in a large, bold font. Below this heading, there is a message: 'Do not leave this page until API key generation and validation are completed.' At the bottom of the main content area, it says 'API Key Generated'. A green progress bar is visible at the very bottom of the page.

Information Technology Laboratory

NATIONAL VULNERABILITY DATABASE

Generating Your API Key

Do not leave this page until API key generation and validation are completed.

API Key Generated

The false-positive decisions are explicit: Angus Activation was matched to a Jakarta Mail SMTP issue, and Hibernate Validator was matched to the unrelated Nu Html Checker issue. The suppression file is versioned and limited to those CVE/component combinations.

Result	Decision
NVD data	Successfully updated and reused from the local/CI cache.
Report formats	HTML and JSON generated for the parent and service modules.
OSS Index	Disabled because the unauthenticated endpoint returned HTTP 401.
Before remediation	43 unique API Gateway CVEs: 15 Critical, 13 High, 14 Medium, 1 Low.
After remediation	Zero unsuppressed CVEs after Tomcat 11.0.25 and reviewed false-positive rules.
Evidence	dependency-check-report.html, dependency-check-report.json and dependency-check-suppressions.xml.

7.6 Workshop 2 completion status

The static-analysis setup is operational and reproducible. SonarCloud with JaCoCo, PITest, Dependency-Check, Maven verification and CI report-upload controls have all been executed. The

justified code remediation identified by SonarCloud was applied, and the dependency baseline now has zero unsuppressed CVEs after the Tomcat patch and reviewed false-positive decisions.

Workshop 2 is complete as a documented analysis and remediation exercise. PITest surviving mutants remain test-strength follow-up items, not unresolved dependency vulnerabilities; the reports preserve them for future test improvement.

8. Deliverable mapping

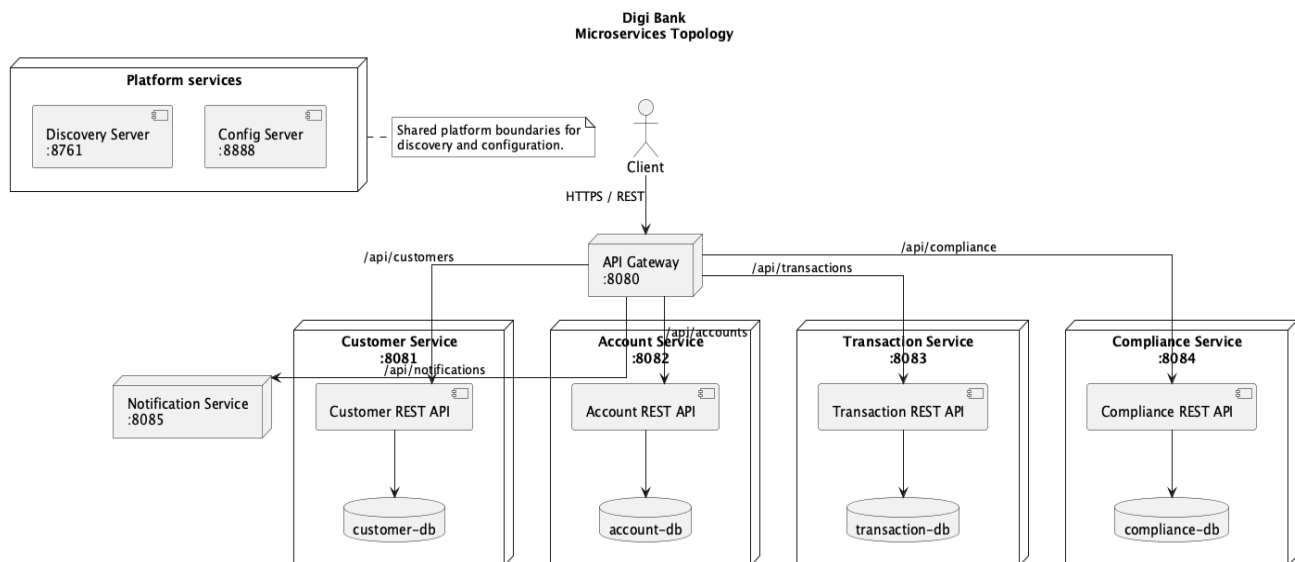
Objective	Demonstrated by
A2.1-A2.2	SAST/Shift Left explanation and source-security review scope.
A2.3	Review of validation, information exposure, dependency and test-strength risks.
A2.4	Maven, SonarCloud, JaCoCo, PITest, Dependency-Check and GitHub Actions.
A2.5	SonarCloud findings interpreted as remediation, accepted decision or follow-up.
A2.6-A2.7	Gateway path hardening, PITest configuration fix and documented security decisions.
A2.8	Passing build/tests, runtime DAST evidence and reproducible static-analysis controls.
A3.1-A3.8	Workshop 1 Newman/ZAP report and runtime evidence retained in this continuation PDF.

The result is a documented chain from source and dependency analysis, to risk interpretation, to remediation, to runtime revalidation.

Workshop 3: Dynamic Security Analysis

This section continues the static-analysis and remediation work with validation against the running DigiBank microservices implementation. DAST observes the behavior visible to an external client; it does not replace the source and dependency analysis already documented above.

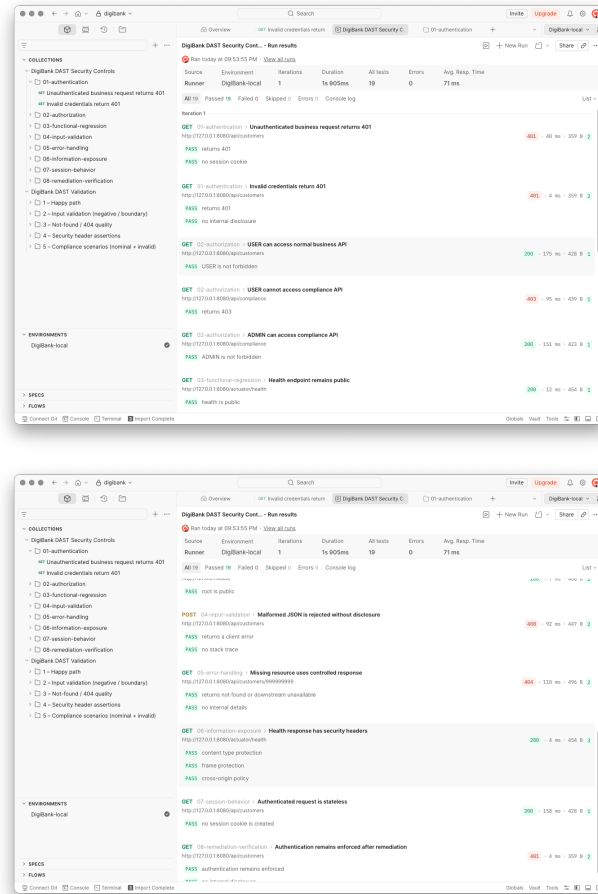
Workshop 2 answered: what weaknesses can be found in the source, dependencies, configuration and tests? Workshop 3 answers: after those weaknesses are fixed, does the running application actually enforce the intended behavior without breaking valid banking flows? A fix can be incorrectly wired, applied to only one route or behave differently after deployment, so runtime revalidation is required.



The API Gateway is the runtime security boundary. All business requests in the campaign enter through this boundary before reaching the internal services.

Workshop 3: Campaign and evidence

The campaign combines manual scenario design in Postman, repeatable Newman assertions and an independent OWASP ZAP scan. The same categories are checked after remediation: authentication, authorization, input validation, error handling, information exposure, session behavior, headers and functional regression.



The Postman results show the expected access decisions and controlled responses. Missing credentials return 401, insufficient roles return 403, and malformed or unknown requests return controlled 4xx responses.

```
08-remediation-verification
Authentication remains enforced after remediation
GET http://127.0.0.1:8080/api/customers [401 Unauthorized, 359B, 6ms]
✓ authentication remains enforced
✓ no internal disclosure
```

	executed	failed
iterations	1	0
requests	12	0
test-scripts	12	0
prerequest-scripts	0	0
assertions	19	0

```
total run duration: 1074ms
total data received: 1.24kB (approx)
average response time: 64ms [min: 4ms, max: 200ms, s.d.: 59ms]
[INF0] Functional report: /Users/adorsys123/dev/digibank-core/dast/reports/202609
5/functional-newman-report.json
```

Newman provides the repeatable command-line validation used locally and in CI.

Workshop 3: Revalidation result

Validation point	Final result
Functional regression	24 requests, 74 assertions, 0 failures.
Security campaign	12 requests, 19 assertions, 0 failures.
Authentication	Missing and invalid credentials return 401.
Authorization	USER is denied protected compliance access; ADMIN is accepted.
Input and errors	Malformed and invalid requests return controlled 4xx responses without internal details.
Session behavior	No session cookie is created.
ZAP	65 passing rules, 0 failures, 2 documented local warnings.

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	1
Low	0
Informational	1
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://127.0.0.1:8080	Percentage of responses with status code 2xx	50 %
Info	Informational	http://127.0.0.1:8080	Percentage of responses with status code 4xx	50 %
Info	Informational	http://127.0.0.1:8080	Percentage of endpoints with content type application/json	33 %
Info	Informational	http://127.0.0.1:8080	Percentage of endpoints with method GET	100 %
Info	Informational	http://127.0.0.1:8080	Count of total endpoints	3

The remaining ZAP warnings are documented local observations: public non-storable resources and the plain-HTTP authentication limitation. They are not evidence of an authentication bypass or cacheable authenticated banking responses.

Workshop 3 is complete when the same runtime scenarios pass after remediation and the remaining scanner observations have an explicit technical decision.

Workshop 4: Securing Containers, Dependencies, and Deployment Artifacts

This workshop examines the artifacts that package and deliver DigiBank. Workshop 2 analyzed application dependencies and source behavior; Workshop 3 validated the running HTTP system; this section checks what is actually built and executed: Maven dependencies, Dockerfiles, runtime images, Compose configuration, repository files and CI controls.

The evidence uses the current Java 21 Spring Boot microservices topology. The assignment source contains older Java 17 and monolith assumptions, so the findings and decisions below are mapped to the actual microservices repository and its Docker Compose delivery model.

Scope and artifact inventory

Artifact under review	Security question
Maven POMs and transitive dependencies	Are known vulnerable components included in the build?
Dockerfiles and runtime images	Does the final image contain unnecessary tools, vulnerable packages or excessive privileges?
Compose configuration	Are ports, networks, credentials, health checks and service boundaries intentional?
Repository and CI workflow	Are secrets and configuration risks detected automatically before delivery?

The implementation uses multi-stage Dockerfiles: Maven and the JDK are used only in the build stage, while the runtime stage contains the packaged service JAR and Java 21 runtime. Compose defines the gateway, business services, platform services and PostgreSQL dependencies. Runtime configuration is supplied through environment variables rather than embedding workshop credentials in source files.

```
⇒ ⇒ transferring context: 4.47kB
⇒ [discovery-server internal] load build context
⇒ ⇒ transferring context: 4.27kB
⇒ CACHED [transaction-service stage-1 2/4] RUN apk upgrade --no-cache
⇒ CACHED [transaction-service stage-1 3/4] WORKDIR /app
⇒ CACHED [transaction-service build 2/13] WORKDIR /workspace
⇒ CACHED [notification-service build 3/13] COPY pom.xml .
⇒ CACHED [notification-service build 4/13] COPY api-gateway/pom.xml api-gateway/pom.xml
⇒ CACHED [notification-service build 5/13] COPY discovery-server/pom.xml discovery-server/pom.xml
⇒ CACHED [notification-service build 6/13] COPY config-server/pom.xml config-server/pom.xml
⇒ CACHED [notification-service build 7/13] COPY customer-service/pom.xml customer-service/pom.xml
⇒ CACHED [notification-service build 8/13] COPY account-service/pom.xml account-service/pom.xml
⇒ CACHED [notification-service build 9/13] COPY transaction-service/pom.xml transaction-service/pom.xml
⇒ CACHED [notification-service build 10/13] COPY compliance-service/pom.xml compliance-service/pom.xml
⇒ CACHED [notification-service build 11/13] COPY notification-service/pom.xml notification-service/pom.xml
⇒ CACHED [notification-service build 12/13] COPY notification-service/src notification-service/src
⇒ CACHED [notification-service build 13/13] RUN mvn -B -pl notification-service -am package -DskipTests
⇒ CACHED [notification-service stage-1 4/4] COPY --from=build /workspace/notification-service/target/notification-service-*.jar app.jar
⇒ [notification-service] exporting to image
⇒ ⇒ exporting layers
⇒ ⇒ exporting manifest sha256:25b1cb734cccec9021a02b398a6fe70555935f5031454d08c388c7a2f9fd68ff
⇒ ⇒ exporting config sha256:4454dffa4859e904d2cea9ed80793d32bb77ad841c174436da191d47c33ba1c
```

The multi-stage build keeps build tooling out of the runtime artifact.

Container scan method

The `make container-scan` command runs Trivy against every locally built application image with vulnerability and secret scanners enabled. It also checks the Docker Compose configuration and the repository filesystem. JSON and SARIF reports are written under `dist/reports/container-scan/`, and High or Critical unfixed findings fail the command. CI preserves the reports through an `always()` artifact-upload step, so a failed security gate does not erase the evidence needed to investigate it.

The generated evidence includes one JSON and one SARIF report per image, plus JSON and SARIF reports for the Compose configuration and the filesystem scan. This means the result is based on the complete artifact set rather than only a terminal summary.

```
✓ Image digibank-microservices-discovery-server Built 221.0
✓ Image digibank-microservices-config-server Built 221.0
✓ Image digibank-microservices-notification-service Built 221.0
✓ Image digibank-microservices-compliance-service Built 221.0
✓ Image digibank-microservices-account-service Built 221.0
✓ Image digibank-microservices-customer-service Built 221.0
✓ Image digibank-microservices-api-gateway Built 221.0
✓ Image digibank-microservices-transaction-service Built 221.0
→ digibank-core git:(main) ✗ make container-scan
/Applications/Xcode.app/Contents/Developer/usr/bin/make -C digibank-microservices container-sc
n
Scanning digibank-microservices-account-service
2026-09-11T12:33:25+01:00 INFO [vuln] Need to update DB
2026-09-11T12:33:25+01:00 INFO [vuln] Downloading vulnerability DB...
2026-09-11T12:33:25+01:00 INFO [vuln] Downloading artifact... repo="mirror.g
r.io/aquasec/trivy-db:2"
112.23 MiB / 112.23 MiB [-----] 100.00% 2.09 MiB p/s 54
2026-09-11T12:34:22+01:00 INFO [vuln] Artifact successfully downloaded repo="f
irror.gcr.io/aquasec/trivy-db:2"
2026-09-11T12:34:22+01:00 INFO [vuln] Vulnerability scanning is enabled
2026-09-11T12:34:22+01:00 INFO [secret] Secret scanning is enabled
2026-09-11T12:34:22+01:00 INFO [secret] If your scanning is slow, please try '--scann
rs vuln' to disable secret scanning
2026-09-11T12:34:22+01:00 INFO [secret] Please see https://trivy.dev/docs/v0.69/guide
scanner/secret#recommendation for faster secret detection
2026-09-11T12:34:25+01:00 INFO [javadb] Downloading Java DB...
2026-09-11T12:34:25+01:00 INFO [javadb] Downloading artifact... repo="mirror.g
r.io/aquasec/trivy-java-db:1"
919.51 MiB / 919.51 MiB [-----] 100.00% 3.94 MiB p/s 3m54
2026-09-11T12:38:20+01:00 INFO [javadb] Artifact successfully downloaded repo="f
irror.gcr.io/aquasec/trivy-java-db:1"
2026-09-11T12:38:20+01:00 INFO [javadb] Java DB is cached for 3 days. If you want to
```

This capture shows the Trivy output used during the container review.

Findings and remediation

The first completed image scan identified five High Alpine package findings in account-service: OpenSSL/libcrypto, OpenSSL/libssl and libexpat. The application JAR contained zero findings. These were container-layer findings, so the appropriate fix was a base-image package update rather than a Maven change.

```
2026-09-11T12:38:21+01:00 INFO [vuln] Vulnerability scanning is enabled
2026-09-11T12:38:21+01:00 INFO [secret] Secret scanning is enabled
2026-09-11T12:38:21+01:00 INFO [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-09-11T12:38:21+01:00 INFO [secret] Please see https://trivy.dev/docs/v0.69/guide/scanner/secret#recommendation for faster secret detection
2026-09-11T12:38:21+01:00 INFO Detected OS family="alpine" version="3.24.1"
2026-09-11T12:38:21+01:00 WARN This OS version is not on the EOL list family="alpine" version="3.24"
2026-09-11T12:38:21+01:00 INFO [alpine] Detecting vulnerabilities... os.version="3.24" repository="3.24" pkg.num=73
2026-09-11T12:38:21+01:00 INFO Number of language-specific files num=1
2026-09-11T12:38:21+01:00 INFO [jar] Detecting vulnerabilities...
2026-09-11T12:38:21+01:00 WARN Using severities from other vendors for some vulnerabilities. Read https://trivy.dev/docs/v0.69/guide/scanner/vulnerability#severity-selection for details.

Report Summary

+-----+-----+-----+-----+
| Target | Type | Vulnerabilities | Secrets |
+-----+-----+-----+-----+
| digibank-microservices-account-service (alpine 3.24.1) | alpine | 5 | - |
| app/app.jar | jar | 0 | - |
+-----+-----+-----+-----+

Legend:
- '*': Not scanned
- '0': Clean (no security findings detected)

digibank-microservices-account-service (alpine 3.24.1)
Total: 5 (HIGH: 5, CRITICAL: 0)

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Library | Vulnerability | Severity | Status | Installed Version | Fixed Version | Title |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| libcrypto3 | CVE-2026-14456 | HIGH | Fixed | 3.5.7-r0 | 3.5.8-r0 | openssl: OpenSSL: Denial of Service via unbounded memory growth in QUIC server... https://www.openssl.com/news/cve-2026-14456 |
| libexpat | CVE-2026-76956 | | | 2.8.3-r0 | 2.8.4-r0 | libexpat: Libexpat: Denial of Service via hash flooding attack with crafted XML... https://www.openssl.com/news/cve-2026-76956 |
| | CVE-2026-76957 | | | | | libexpat: Libexpat: Memory corruption vulnerability allows arbitrary code execution on detail of... https://www.openssl.com/news/cve-2026-76957 |
| libssl3 | CVE-2026-14456 | | | 3.5.7-r0 | 3.5.8-r0 | openssl: OpenSSL: Denial of Service via unbounded memory growth in QUIC server... https://www.openssl.com/news/cve-2026-14456 |
| openssl | | | | | | |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

make[1]: *** (container-scan) Error 1
Report Summary

+-----+-----+-----+-----+
| Target | Type | Vulnerabilities | Secrets |
+-----+-----+-----+-----+
| digibank-microservices-account-service (alpine 3.24.1) | alpine | 0 | - |
| app/app.jar | jar | 0 | - |
+-----+-----+-----+-----+

Legend:
- '*': Not scanned
- '0': Clean (no security findings detected)

Scanning digibank-microservices-api-gateway
2026-09-11T12:42:25+01:00 INFO [vuln] Vulnerability scanning is enabled
2026-09-11T12:42:25+01:00 INFO [secret] Secret scanning is enabled
2026-09-11T12:42:25+01:00 INFO [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-09-11T12:42:25+01:00 INFO [secret] Please see https://trivy.dev/docs/v0.69/guide/scanner/secret#recommendation for faster secret detection
2026-09-11T12:42:25+01:00 INFO Detected OS family="ubuntu" version="26.04"
2026-09-11T12:42:25+01:00 WARN This OS version is not on the EOL list family="ubuntu" version="26.04"
2026-09-11T12:42:25+01:00 INFO [ubuntu] Detecting vulnerabilities... os.version="26.04" pkg.num=140
2026-09-11T12:42:25+01:00 INFO Number of language-specific files num=2
2026-09-11T12:42:25+01:00 INFO [golang] Detecting vulnerabilities...
2026-09-11T12:42:25+01:00 INFO [jar] Detecting vulnerabilities...
2026-09-11T12:42:25+01:00 WARN Using severities from other vendors for some vulnerabilities. Read https://trivy.dev/docs/v0.69/guide/scanner/vulnerability#severity-selection for details.
```

The account-service before-and-after evidence shows the container-layer findings being addressed.

The API Gateway image was clean at the Java application layer but included the unrelated `pebble` utility in its Ubuntu runtime layer, producing eight High findings in bundled Go components. The same issue was present in discovery-server and config-server. Their runtime images were moved to Java 21 Alpine, Alpine packages were updated, and the unrelated utility was removed. All service images also run as the non-root `digibank` user.

Report Summary

Target	Type	Vulnerabilities	Secrets
digibank-microservices-api-gateway (Ubuntu 20.04)	ubuntu	0	-
app/app.jar	jar	0	-
usr/bin/pssble	go-binary	0	-

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

usr/bin/pssble (go-binary)

Total: 0 (HIGH: 0, CRITICAL: 0)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
stdlib	CVE-2020-33818	High	Fixed	v1.20.5	1.20.13, 1.20.6, 1.27.0-rc.3	encoding/gof: goLang Go encoding/gof: Denial of Service via excessive recursion in Unmarshal... https://ask-ubuntu.com/feeds/CVE-2020-33818
	CVE-2020-39821					golang.org/x/net/dns: goLang net/http: goLang.org/x/net/dns: Privilege escalation via incorrect Punycode Label processing https://ask-ubuntu.com/feeds/CVE-2020-39821
	CVE-2020-46000				1.20.6, 1.27.0-rc.3	golang.org/x/net/dns/dnsmessag: goLang.org/x/net/dns/dnsmessag: Denial of Service via Invalid DNS record parsing https://ask-ubuntu.com/feeds/CVE-2020-46000
	CVE-2020-56853				1.20.13, 1.20.6, 1.27.0-rc.3	net/http: goLang Go net/http: Unencrypted HTTP/2 connections vulnerable to Denial of Service... https://ask-ubuntu.com/feeds/CVE-2020-56853
	CVE-2020-56858					html/template: goLang Go html/template: Cross-site Scripting via pathological Input https://ask-ubuntu.com/feeds/CVE-2020-56858
	CVE-2020-56859					encoding/xml: goLang Go: Denial of Service via XML decoding recursion depth issues... https://ask-ubuntu.com/feeds/CVE-2020-56859
	CVE-2020-56860					net/url: goLang goLang net/url: Denial of Service from Unmarshal complexity in path... https://ask-ubuntu.com/feeds/CVE-2020-56860
	CVE-2020-56862					crypto/tls: goLang GoLang crypto/tls: Denial of Service via indefinite recursive messages https://ask-ubuntu.com/feeds/CVE-2020-56862

Report Summary

Target	Type	Vulnerabilities	Secrets
digibank-microservices-api-gateway (alpine 3.24.1)	alpine	0	-
app/app.jar	jar	0	-

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

Scanning digibank-microservices-compliance-service

2024-09-11T12:49:01-01:00 INFO [w/o] Vulnerability scanning is enabled

2024-09-11T12:49:01-01:00 INFO [secret] Secret scanning is enabled

2024-09-11T12:49:01-01:00 INFO [secret] If your scanning is slow, please try "-scanners w/o" to disable secret scanning

2024-09-11T12:49:01-01:00 INFO [secret] Please see <https://trivy.dev/docs/0609/guide/scanner/secret/recommendation> for faster secret detection

2024-09-11T12:49:01-01:00 INFO [detected] family="alpine" version="3.24.1"

2024-09-11T12:49:01-01:00 WARN This OS version is not on the EOL list family="alpine" version="3.24.1"

2024-09-11T12:49:01-01:00 INFO [info] [

The gateway evidence shows that the remediation changed the runtime artifact rather than hiding scanner results or changing banking logic.

Final artifact validation

The complete post-remediation scan covered all eight application images: API Gateway, customer-service, account-service, transaction-service, compliance-service, discovery-server, config-server and notification-service. Each image reported zero High and Critical vulnerabilities, zero Java archive findings and no detected secrets. Compose reported zero misconfigurations and zero secrets.

Report Summary				
Target	Type	Vulnerabilities	Secrets	Misconfigurations
account-service/pom.xml	pom	0	-	-
api-gateway/pom.xml	pom	0	-	-
compliance-service/pom.xml	pom	0	-	-
config-server/pom.xml	pom	0	-	-
customer-service/pom.xml	pom	0	-	-
discovery-server/pom.xml	pom	0	-	-
notification-service/pom.xml	pom	0	-	-
pom.xml	pom	0	-	-
transaction-service/pom.xml	pom	0	-	-
account-service/Dockerfile	dockerfile	-	-	0
api-gateway/Dockerfile	dockerfile	-	-	0
compliance-service/Dockerfile	dockerfile	-	-	0
config-server/Dockerfile	dockerfile	-	-	0
customer-service/Dockerfile	dockerfile	-	-	0
discovery-server/Dockerfile	dockerfile	-	-	0
notification-service/Dockerfile	dockerfile	-	-	0
transaction-service/Dockerfile	dockerfile	-	-	0

Legend:
- '-': Not scanned

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
alpine:latest	28bd5fe8b56d	14.6MB	4.27MB	
budget-wise-online-recovery-server:latest	106a94d3f600	1.07GB	199MB	U
budget-wise-online-server:latest	6479ba7ff3d3	1.08GB	200MB	U
digibank-microservices-account-service:latest	23e46c056b8a	401MB	125MB	
digibank-microservices-api-gateway:latest	04503ac868cd	348MB	100MB	
digibank-microservices-compliance-service:latest	8eb1982d8418	401MB	125MB	
digibank-microservices-config-server:latest	e7d60c5c73a1	337MB	95MB	
digibank-microservices-customer-service:latest	93c2e0994661	401MB	125MB	
digibank-microservices-discovery-server:latest	a222f1b93e96	337MB	95MB	
digibank-microservices-notification-service:latest	fcfc72a43158	344MB	98.2MB	
digibank-microservices-transaction-service:latest	a2b026e47d22	401MB	125MB	
digibank-parent-app:latest	791ca1555d15	403MB	128MB	U
digibank-parent-wildfly:latest	4fcdfc39bc8a	1.47GB	544MB	U

The final Trivy report and image-size evidence show the clean result and the delivered artifact inventory.

The filesystem scan reported no High or Critical findings and no secrets. It retained eight Low `DS-0026` observations because the individual Dockerfiles do not declare Docker `HEALTHCHECK` instructions. This remains visible and is recorded as an accepted low-risk observation for the workshop: Compose checks cover databases and platform services, while smoke and stack-validation commands verify business-service readiness. The observation is not suppressed.

CI controls and deployment decisions

The CI container-security job rebuilds the images, installs Trivy, runs the same container scan and uploads the generated JSON and SARIF reports. This keeps container checks separate from Maven SAST and runtime DAST while scanning the same build artifacts that are used for validation.

The local Compose deployment intentionally publishes several ports for workshop inspection. In a production deployment, only the gateway would be public, database ports would remain internal, credentials would come from a secret-management system, and images would run with a dedicated non-root runtime user. These are documented deployment decisions, not claims that the single-host educational environment is production infrastructure.

Report Summary

Target	Type	Vulnerabilities	Secrets
digibank-microservices-notification-service (alpine 3.24.1)	alpine	0	-
app/app.jar	jar	0	-

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

Scanning digibank-microservices-transaction-service

```
2026-09-11T12:42:59+01:00 INFO [vuln] Vulnerability scanning is enabled
2026-09-11T12:42:59+01:00 INFO [secret] Secret scanning is enabled
2026-09-11T12:42:59+01:00 INFO [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-09-11T12:42:59+01:00 INFO [secret] Please see https://trivy.dev/docs/v0.69/guide/scanner/secret#recommendation for faster
ecret detection
2026-09-11T12:43:01+01:00 INFO Detected OS family="alpine" version="3.24.1"
2026-09-11T12:43:01+01:00 WARN This OS version is not on the EOL list family="alpine" version="3.24"
2026-09-11T12:43:01+01:00 INFO [alpine] Detecting vulnerabilities... os_version="3.24" repository="3.24" pkg_num=73
2026-09-11T12:43:02+01:00 INFO Number of language-specific files num=1
2026-09-11T12:43:02+01:00 INFO [jar] Detecting vulnerabilities...
```

🔔 **Notices:**
- Version 0.74.0 of Trivy is now available, current version is 0.69.3

To suppress version checks, run Trivy scans with the --skip-version-check flag

```
2026-09-11T12:43:02+01:00 INFO [vuln] Vulnerability scanning is enabled
2026-09-11T12:43:02+01:00 INFO [secret] Secret scanning is enabled
2026-09-11T12:43:02+01:00 INFO [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-09-11T12:43:02+01:00 INFO [secret] Please see https://trivy.dev/docs/v0.69/guide/scanner/secret#recommendation for faster
ecret detection
2026-09-11T12:43:02+01:00 INFO Detected OS family="alpine" version="3.24.1"
2026-09-11T12:43:02+01:00 WARN This OS version is not on the EOL list family="alpine" version="3.24"
2026-09-11T12:43:02+01:00 INFO [alpine] Detecting vulnerabilities... os_version="3.24" repository="3.24" pkg_num=73
2026-09-11T12:43:02+01:00 INFO [jar] Detecting vulnerabilities... num=1
2026-09-11T12:43:02+01:00 INFO [vuln] Vulnerability scanning is enabled
2026-09-11T12:43:02+01:00 INFO [secret] Secret scanning is enabled
2026-09-11T12:43:02+01:00 INFO [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-09-11T12:43:02+01:00 INFO [secret] Please see https://trivy.dev/docs/v0.69/guide/scanner/secret#recommendation for faster
ecret detection
```

Assah Bismark / Digibank Microservices / Issues

Issues master

Filters Sort by Priority

Applied filters (1) Status: Open Clear filters

Select issues for bulk actions 29 issues 3h 9min effort

api-gateway/.../main/java/com/digibank/gateway/SecurityConfig.java

☐ Make sure disabling Spring Security's CSRF protection is safe here. Consistency

Security High cwe former-hotspot ...

Open Assah Bismark L20 • 5min effort • 2 hours ago

account-service/.../com/digibank/account/error/ApiError.java

☐ Explicitly specify the time zone by passing a ZoneId or a Clock to the .now() method. Intentionality

Reliability Info Maintainability Info datetime java8 ...

Open Assah Bismark L7 • 5min effort • 12 hours ago

api-gateway/.../main/java/com/digibank/gateway/GatewayExceptionHandler.java

☐ Explicitly specify the time zone by passing a ZoneId or a Clock to the .now() method. Intentionality

Reliability Info Maintainability Info datetime java8 ...

Open Assah Bismark L25 • 5min effort • 12 hours ago

The supporting scan captures are read together with the generated JSON and SARIF reports, which remain the machine-readable evidence.

Deliverable mapping A4.1-A4.8

Objective	Demonstrated by
A4.1	Risks in dependencies, image layers and container configuration are explained.
A4.2	Dockerfiles, Compose, Maven POMs, images, environment configuration and CI workflow are inventoried.
A4.3	Dependency-Check and Trivy analyze declared, transitive and image-embedded components.
A4.4	Multi-stage Dockerfiles, runtime bases, image contents, ports and runtime parameters are reviewed.
A4.5	Dependency updates, Alpine package updates, image reduction and non-root execution are applied.
A4.6	Each remediation is connected to its risk reduction and verified by post-remediation scanning.
A4.7	GitHub Actions rebuilds images, runs container-scan and uploads JSON/SARIF reports.
A4.8	The report, screenshots, generated reports and reproducible commands provide the final deliverable.

Workshop 4 completes the delivery-artifact security chain: application analysis, runtime validation, container hardening and automated artifact verification.